



EngX™



EPAM.AI

GenAI Intensive Program

Kickstart Your AI-Powered Modernization

with Claude Code



Masterclass #3

Claude Code:

Common Development Workflows

AI Masterclass Outline

Session #3:

Common Development Workflows

Learning Objectives

1. Automate routine development tasks
2. Generate comprehensive tests and documentation
3. Create custom tooling for project-specific needs
4. Establish efficient daily workflows

Topics Covered

1. Code Generation
2. Test Generation
3. Code Documentation
4. Code Refactoring
5. Tooling Generation
6. Working With Git
7. Bug Fixing
8. Problem Solving
9. Deep Research

Effective Prompting Patterns

✗ Weak Prompts

- “Fix this bug”.
- “Write tests”.
- “Refactor this code”.
- “Add documentation”.

💡 *Describe the goal, not the solution. Provide constraints.
Reference existing patterns in the codebase.*

✓ Strong Prompts

- “Fix the null check in parseUser that crashes when email is missing”.
- “Add unit tests for OrderService covering edge cases: empty cart, expired discount, null user”.
- “Extract the validation logic from createOrder into a separate OrderValidator class”.
- “Add JSDoc to all public methods in src/api/ following existing patterns in src/utils/”.

Common Pitfalls & Antipatterns

- **Vague prompts** – “Fix this” without specifying what’s broken or where.
- **Skipping Plan Mode for large changes** – Leads to sprawling, hard-to-review diffs.
- **Not running tests before/after** – Refactoring without test safety net risks regressions.
- **Accepting generated code without review** – Always verify AI output, especially for security-sensitive code.
- **Overloading a single session** – Mixing unrelated tasks degrades context quality; use one session per logical unit.
- **Empty CLAUDE.md** – Missing project context means Claude guesses your conventions instead of following them.
- **Ignoring permission prompts** – Auto-accepting everything defeats safety; configure allowedTools in settings.json instead.

Multi-file Context

How Claude Code Navigates Your Codebase

- Automatically searches with Glob (find files) and Grep (search content).
- Reads related files to understand context before making changes.
- Spawns sub-agents (Task tool) for parallel exploration of large codebases.

Help Claude Focus

- Use @file to point Claude at specific files when you know where the problem is.
- Break large tasks into scoped prompts when context window gets full.
- Scope requests to specific directories: “Only modify files in src/api/”.
- Use CLAUDE.md to list key files and architecture so Claude starts with the right context.

Context Management & Session Strategy

When to Start a Fresh Session

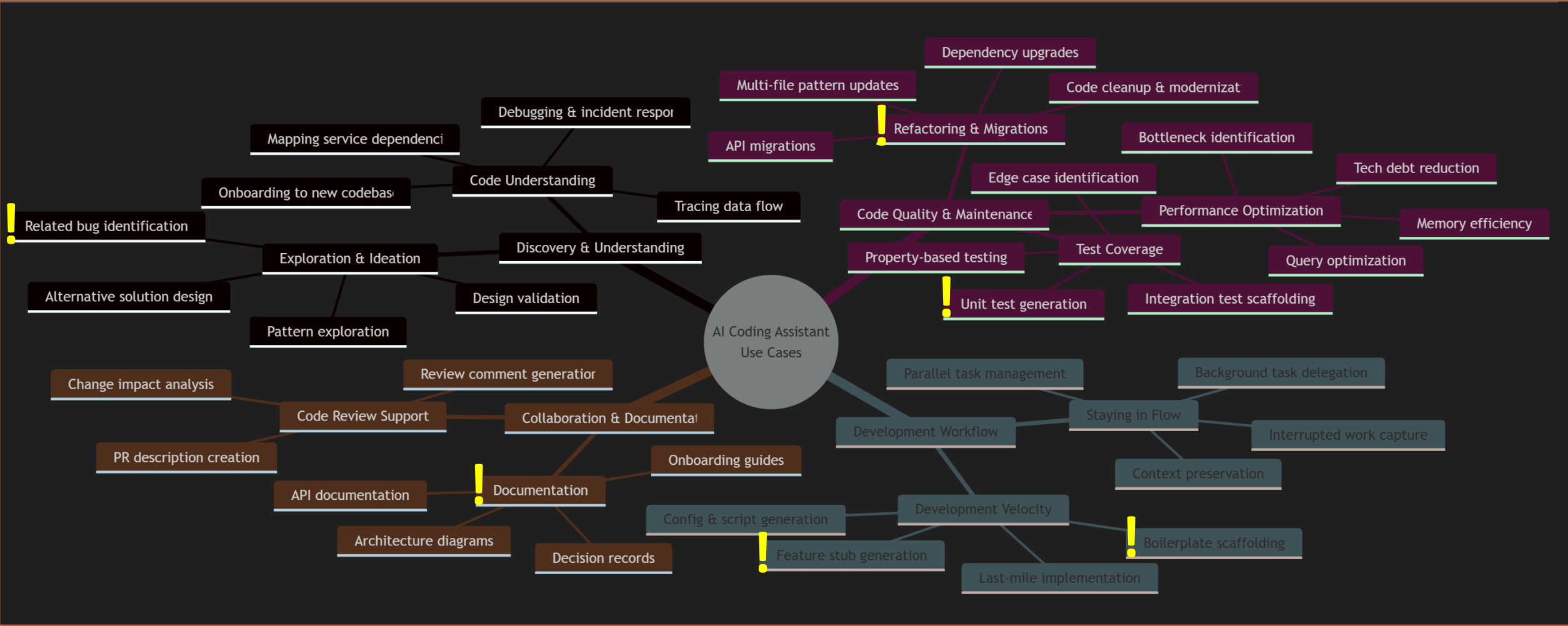
- Switching to a completely unrelated task or different part of the codebase.
- Claude starts making mistakes or “forgetting” earlier instructions.
- After completing a logical unit of work (commit boundary).

Staying Productive in Long Sessions

- `/compact` – Summarize conversation history to reclaim context window space.
- Re-anchor – If Claude loses the thread, restate your goal and current state explicitly.
- One task per session – Keep sessions scoped to one logical unit for best results.
- `claude --continue` – Resume the most recent conversation if you accidentally closed the terminal.

AI-Assisted Use Cases

- Code Generation
- Code Documentation
- Code Refactoring
- Test Generation
- Tooling Generation
- Working with Git
- Bug Fixing



Boilerplate Generation

AI-assisted generation of repetitive, standard code structures and patterns to accelerate development.

Focuses on creating consistent scaffolding code, data models, API endpoints, and framework-specific boilerplate that follows established conventions and patterns.

Use Skills for Boilerplate Generation

```

  skills
    fastapi-crud
      assets
        templates
          exceptions_template.py
          router_template.py
          schemas_template.py
      references
        exceptions.md
        logging.md
        schemas.md
    SKILL.md
```

****Create New Skill:****

```
...
```

Use skill-creator to help me create a new skill for generating our standard FastAPI endpoints. Our pattern includes:

- Router with APIRouter and proper tags
- Pydantic schemas with field validation
- Dependency injection for DB sessions via Depends()
- Custom exception handlers returning RFC 7807 responses
- Structured logging with structlog
- OpenAPI documentation with examples

Create the skill structure in `.claude/skills/fastapi-crud/`

```
...
```

```
...
```

Use the fastapi-service skill to create a new microservice called 'payment-gateway' with our standard structure including health check endpoints, Prometheus metrics, and circuit breaker patterns using the tenacity library.

```
...
```

Features Implemented

Feature	Implementation
Router	APIRouter with /products prefix and proper tags
Schemas	ProductCreate, ProductUpdate, ProductResponse, PaginatedProductResponse
Validation	Field validators for name, price (max 2 decimals), stock (non-negative)
DB Session	Dependency injection via Depends(get_db)
Exceptions	RFC 7807 responses (NotFoundError, ConflictError)
Logging	Structured logging with structlog on all operations
OpenAPI	Full documentation with examples

API Endpoints

Method	Endpoint	Description
GET	/api/v1/products/	List products (paginated, filterable)
GET	/api/v1/products/{id}	Get single product
POST	/api/v1/products/	Create product
PUT	/api/v1/products/{id}	Full update
PATCH	/api/v1/products/{id}	Partial update
DELETE	/api/v1/products/{id}	Delete product

Running the API

```
# Install dependencies
pip install fastapi uvicorn sqlalchemy structlog

# Run the server
uvicorn app.main:app --reload

Then visit http://localhost:8000/docs for the interactive API documentation.
```

Features with Plan Mode

Iterative development using Claude Code's plan mode for review before implementation. Enables human-in-the-loop validation of AI-generated implementation strategies.

 [cc-handbook/fundamentals/extended-plan-mode](#)

Plan mode examples

```
```
```

[Shift+Tab for Plan Mode]

I want to implement a background task processing system:

- Celery with Redis as broker
- Task retry logic with exponential backoff
- Dead letter queue for failed tasks
- Task monitoring dashboard
- Integration with our existing FastAPI app

Create a plan that considers our current architecture in src/ and proposes the integration approach.

```
```
```

```
```
```

[Press Shift+Tab to enter Plan Mode, then:]

I need to implement a complete user authentication system

- Email/password login using passlib for hashing
- OAuth integration (Google, GitHub) via authlib
- JWT token management with python-jose
- Refresh token rotation
- Password reset flow with email via SendGrid
- Rate limiting using slowapi

Analyze our current FastAPI codebase and create a detailed implementation plan before writing any code.

```
```
```

Plan mode output

``markdown

Implementation Plan: User Authentication System

Phase 1: Foundation (files: 4, estimated changes: ~200 lines)

- Create auth module structure in src/auth/
- Set up JWT configuration with python-jose
- Implement password hashing with passlib[bcrypt]

Phase 2: Core Authentication (files: 6, estimated changes: ~400 lines)

- User registration with email verification
- Login endpoint returning JWT tokens
- Token refresh mechanism with rotation

Phase 3: OAuth Integration (files: 5, estimated changes: ~300 lines)

- Google OAuth via authlib
- GitHub OAuth integration
- OAuth callback handlers

Phase 4: Security Features (files: 4, estimated changes: ~250 lines)

- Password reset flow with secure tokens
- Email sending via SendGrid
- Rate limiting middleware with slowapi

Dependencies to Add to pyproject.toml:

- python-jose[cryptography]
- passlib[bcrypt]
- authlib
- slowapi

Shall I proceed with Phase 1?

Code Generation: Readiness Checklist

- ✓ **CLAUDE.md with project context** – Tech stack, architecture, coding conventions, key file pointers – Claude reads this every session; keep under 200 lines.
- ✓ **Golden example files in codebase** – 2–3 reference implementations for each pattern type – “Follow exactly this pattern from UserService.ts” beats any prose description.
- ✓ **Project spec with data models and API contracts** – Link from CLAUDE.md to docs/project-specification.md – AI brainstorms better with written specs than verbal descriptions.
- ✓ **Slash commands for repeatable scaffolding** – Package generation tasks as /commands (e.g., /new-endpoint) with post-generation checklists (register in DI, run tests, update routes).
- ✓ **Naming conventions and variable schemas** – File naming, class naming, directory structure – without these, each AI generation follows different patterns.
- ✓ **Plan Mode for features (Shift+Tab)** – Explore → Plan → Implement → Commit; use the interview technique for complex features to generate SPEC.md.
- ✓ **Verification mechanism defined** – #1 rule: give AI a way to verify its own work – tests, linter, expected output; without this, code “looks right but doesn’t work” (Anthropic).

Code Documentation

AI-assisted code documentation that enhances code readability, maintainability, and knowledge transfer across development teams.

This automated approach ensures consistent, comprehensive documentation that helps onboard new team members and reduces the time spent understanding unfamiliar code.

💡 *Use it not only for initial generation but also to maintain existing docs*

Code Documentation

Use slash command

```
```
```

Generate a comprehensive README.md for this Python project that includes:

- Project overview and purpose
- Prerequisites (Python version, system dependencies)
- Installation with pip/poetry/uv
- Configuration options (document all env variables from .env.example)
- API endpoints summary (extract from FastAPI routes)
- Development workflow (running locally, tests, linting)
- Deployment guide

Base this on the actual code structure, not assumptions.

```
```
```

```
```
```

Review the changes in the last 5 commits and update any documentation that's now out of sync. Check:

- README.md
- All docstrings in modified files
- API documentation in routers
- Type hints accuracy

Show me what needs updating before making changes.

```
```
```

Code Documentation

```

/
2. Add deprecated flag template for future use in router responses
3. app/routers/products.py:51-55 - Consider adding examples to Query parameters:
search: Optional[str] = Query(
    None,
    description="Search by product name",
    example="headphones"
),

```

Summary Dashboard

Area	Status	Issues Found	Priority
-----	-----	-----	-----
README.md	 Missing	1	 High
Docstrings	 Excellent	2 minor	 Low
Type Hints	 Comprehensive	2 minor	 Low
FastAPI Docs	 Excellent	3 minor	 Low

Recommended Actions

High Priority

1. **Create README.md** - Essential for project onboarding

Low Priority (Polish)

2. Enhance lifespan() docstring with Args/Yields sections
3. Expand Product model class docstring with Attributes
4. Add return type hint to exception handler
5. Add example parameter to Query definitions

Would you like me to proceed with implementing any of these updates?

Documentation Tiers

HOT — Always Loaded (<60 lines)

- CLAUDE.md — Pointers only: links to constitution, project spec, commands.
- Loaded every session. Keep it thin. Never duplicate content here.

WARM — On-Demand by Agent/Workflow

- docs/constitution.md — Non-negotiable rules.
- docs/project-specification.md — Living spec (architecture, API, conventions).
- docs/[domain].md — Scoring, layers, services, operations.

COLD — Archived, Reference Only

- output/planning-artifacts/ — PRDs, architecture (frozen after implementation).
- output/implementation-artifacts/archive/ — Completed stories.

💡 *Source: arXiv “Codified Context” paper + ETH Zurich study on AI agent context*

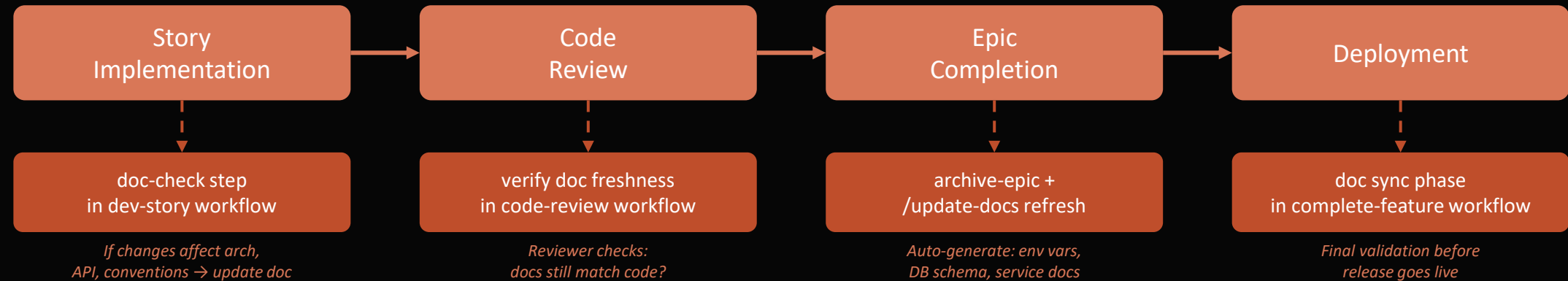
AI Docs: Key Principles

- **Single Source of Truth**
Each fact lives in exactly one place.
Docs reference code, never duplicate it.
- **Living Specs vs. Frozen Artifacts**
docs/ = living specs that evolve with code.
Planning artifacts = frozen after implementation.
- **Agents Don't Update Docs Unprompted**
Every study confirms this.
You MUST add explicit instructions in workflows.
- **Auto-Generate What You Can**
DB schema from ORM, env vars from .env.example,
API docs from router definitions.
- **Validate Freshness in CI**
Check referenced files still exist.
Flag docs older than 30 days.

💡 *Anti-pattern: LLM-generated context files hurt performance by 2-3% vs. no file at all (ETH Zurich)*

Doc Updates in SDLC

Trigger-Based Updates (not scheduled)



CI Pipeline: freshness checks on every PR | Verify referenced files exist | Flag stale docs (>30 days) | /update-docs (manual refresh)

💡 *Key insight: triggered updates > scheduled updates. Docs drift when decoupled from the workflow.*

Code Documentation: Readiness Checklist

- ✓ **Doc format standards in CLAUDE.md** – JSDoc/docstrings/ADRs per file type; explicit “Why over What” rule – “Write comments explaining WHY, not WHAT”.
- ✓ **Documentation tiers configured** – HOT (CLAUDE.md pointers <60 lines), WARM (living specs in docs/), COLD (frozen artifacts) – prevents context window waste.
- ✓ **Well-documented functions as reference** – 2–3 exemplary functions with proper docs – AI matches your style; without examples it generates verbose boilerplate nobody reads.
- ✓ **CI validation for doc freshness** – Verify referenced files still exist, flag docs older than 30 days – stale docs mislead more than no docs at all.
- ✓ **Trigger-based update workflow** – Doc updates after story completion, code review, deployment – not on a schedule; decoupled docs always drift.
- ✓ **Business context documented separately** – AI documents WHAT code does accurately but cannot reliably document WHY business decisions were made – you must provide this context.

Code Refactoring

AI-assisted code refactoring that improves software quality and maintainability by identifying code smells, duplications, and architectural issues.

Claude Code analyzes codebases to suggest targeted improvements, restructure complex logic, extract reusable components, and modernize legacy patterns.

 *Use Plan Mode to select a refactoring approach. Start from principles and goals and drill down to details*

Code Refactoring

1. Run tests first.
2. Enter Plan Mode `Shift+Tab`.
3. Describe goal, not solution.
4. Review approach options - trade-offs.
5. Approve incremental plan - small, testable steps.
6. Run tests after each step - behavior unchanged.
7. Commit incrementally - One commit per logical change.

```
...
```

[Shift+Tab for Plan Mode]

Goal: Reduce coupling between our order processing and inventory management modules.

Current problem: OrderService directly imports and calls InventoryService methods, making them hard to test independently.

Analyze the current coupling and propose refactoring approaches. Consider:

- Dependency injection with FastAPI's Depends
- Event-driven decoupling with signals or message queue
- Abstract base classes / Protocols

Don't change code yet – present options with trade-offs.

```
...
```

```
...
```

src/legacy/data_processor.py uses old Python 2 patterns.

Modernize it to Python 3.11+ standards:

- Replace % formatting with f-strings
- Use pathlib instead of os.path
- Add type hints throughout
- Use dataclasses or Pydantic instead of plain dicts
- Replace threading with asyncio where appropriate
- Follow our current coding standards in src/services/

Ensure all existing tests still pass after refactoring.

```
...
```

Code Refactoring: Readiness Checklist

- ✓ **Passing test suite as safety net** – Non-negotiable: run ALL tests before refactoring – teams report 60% fewer regressions with proper test coverage on target code.
- ✓ **Code quality baseline measured** – Cyclomatic complexity, linter warnings before starting – you need a “before” to prove the refactoring actually improved things.
- ✓ **Defensive pattern awareness in CLAUDE.md** – Document WHY circuit breakers, retries, rate limits exist – AI can break production by removing safeguards it sees as “unnecessary code”.
- ✓ **Plan Mode always on (Shift+Tab)** – Review approach AND blast radius before execution – a bad refactoring plan is worse than no refactoring.
- ✓ **Incremental commits with tests after each** – One logical change per commit, tests green after each step – makes git bisect possible; teams see 40% faster review cycles.
- ✓ **Scope explicitly defined** – Specific files, modules, or patterns – unbounded “refactor this codebase” leads to massive unreviewable diffs.
- ✓ **Rollback plan ready** – Feature flags for large refactors; know the git SHA to revert to – deploy behind flags with shadow traffic if risky.

Test Generation

AI-assisted test generation that automatically creates comprehensive test suites for your codebase.

Claude Code analyzes code to identify critical test scenarios, uncover edge cases, generate realistic test data, and write proper assertions - all while seamlessly integrating with your existing testing frameworks and maintaining consistency with your project's testing patterns.

Test Generation

Use slash command

...

Generate comprehensive pytest tests for `src/services/payment_service.py`:

- Unit tests for all public methods
- Use pytest fixtures for common setup
- Parametrized tests for edge cases (None, empty, boundary values)
- Mock external dependencies (Stripe API, database)
- Test async functions with `pytest-asyncio`
- Error scenarios (network failures, invalid cards, timeouts)

Use our existing test patterns from `tests/` and aim for >90% coverage.

...

Run `pytest --cov=src --cov-report=term-missing` and identify untested code paths in `src/services/`. For each uncovered path:

1. Explain why it matters
2. Generate appropriate tests
3. Verify the new tests pass

Focus on critical business logic first.

...

Test Generation:

Readiness Checklist

- ✓ **Testing framework configured in CLAUDE.md** – Framework name, config file, run command, and key rule: “prefer testing behavior over implementation details”.
- ✓ **Existing test examples as patterns** – 2–3 well-written tests per type – Claude matches your style; without examples it generates textbook tests with excessive mocking.
- ✓ **TDD workflow: tests first, then implementation** – Ask AI to write failing tests first, then implement the fix – most effective pattern per Axur engineering research.
- ✓ **Edge cases listed in the prompt upfront** – Null, empty, boundary, auth failures, concurrency – list them explicitly; AI over-tests happy paths and under-tests error paths if not directed.
- ✓ **Mocking boundaries documented** – Which deps to mock vs. real; specify in CLAUDE.md – prevents brittle tests that break on every refactoring.
- ✓ **Coverage targets on critical paths** – Focus on critical business paths, not line coverage – CI should reject PRs below threshold on changed code.
- ✓ **Know AI testing limits** – AI excels at regression/API tests and edge cases humans skip; weak at exploratory testing and security threat modeling.

Bug Fixing

Claude Code accelerates bug fixing by quickly analyzing error messages and codebase context to identify root causes across multiple files.

It automates investigation by searching patterns, reviewing git history, and examining tests, then generates targeted fixes and validates them by running builds and test suites.

This allows developers to focus on problem-solving rather than manual code navigation and repetitive commands.

Bug Fixing

Use sub agent

...

ultrathink

We have an intermittent bug: some async tasks are being processed twice, resulting in duplicate database entries. It happens roughly 1 in 1000 tasks.

Relevant info:

- Using Celery with Redis broker
- Task acks_late=True for reliability
- Multiple workers running
- Started after scaling to 3 worker instances

Deeply analyze potential race conditions, task acknowledgment issues, or idempotency gaps. Check our task retry logic and database transaction handling with SQLAlchemy.

...

<https://nikiforovall.blog/claude-code-rules/component-reference/agents/root-cause-analyst>

Bug Fixing: Readiness Checklist

- ✓ **Structured bug report format** – Environment + symptom + error + affected files + repro steps – this format vs. “it’s broken” is the #1 factor in fix quality.
- ✓ **Error logs and stack traces in the prompt** – Paste actual errors, not screenshots – Claude traces call stacks across files; structured text > images for code analysis.
- ✓ **Bug Reproduction Test (BRT) written first** – Google research: providing a failing-to-passing test fixes 30% more bugs with 50% fewer steps; 86% of AI-generated BRTs are production-ready.
- ✓ **Git history accessible for bisect** – Claude uses git log/blame to find the breaking commit – add “STARTED AFTER: commit abc123” to narrow the search.
- ✓ **Scope narrowed with @file references** – Point Claude at suspected module – too much context degrades performance; relevant files only, not the entire codebase.
- ✓ **Two-attempt rule** – If stuck after 2 fix attempts: switch model, break problem smaller, or provide more context – don’t loop on the same approach.
- ✓ **Regression test plan defined** – After fix: original test passes + adjacent tests pass + CI green – explicit “fix root cause, don’t suppress the error”.

Problem Solving / Ideation / Rubber Ducking

Key Success Factors

- **Open Dialogue:** Willingness to explore ideas without immediate judgment.
- **Iterative Process:** Multiple rounds of discussion to refine understanding.
- **Specific Questions:** Concrete areas to explore rather than vague requests.
- **Active Engagement:** Building on AI responses with follow-up questions and refinements.

Best Practices

- ✓ **Start Broad, Then Narrow:** Begin with general exploration, then focus on specific aspects .
- ✓ **Ask "What If" Questions:** Explore scenarios and edge cases through hypotheticals.
- ✓ **Challenge Assumptions:** Use AI to question your initial premises and constraints.
- ✓ **Think Out Loud:** Verbalize your thought process to identify gaps or inconsistencies.
- ✓ **Iterate and Refine:** Build on previous responses to deepen understanding.

Common Pitfalls

- ✗ Not providing enough context for meaningful discussion.
- ✗ Not engaging with AI responses or asking follow-up questions.
- ✗ Focusing on getting "the answer" rather than exploring the problem space.

Deep Research

Key Benefits

- ✓ Research with multiple sources
- ✓ Connected findings, patterns, and deeper understanding
- ✓ Organized presentation of findings

Context

Comprehensive research and analysis tasks where AI performs thorough, multi-perspective investigation of complex topics.

Utilizes advanced reasoning techniques and multiple AI tools to provide exhaustive information gathering and analysis.

- 💡 Use **Claude AI** for deep search.
- 💡 Use **@deep-research-agent** to perform a deep search.
- 💡 Use **context7 MCP** to get latest documentation without overly relying on **WebSearch**.

Tooling Generation

Generation of scripts and utilities for development tasks, operations automation, and project-specific tooling. Covers everything from database operations to data processing utilities and deployment automation.

Tooling Generation

1. Generate tool - ``"Create a script that..."``.
2. Test and refine - ``"Add error handling for X"``.
3. Document usage - ``"Add --help documentation"``.
4. Convert to Skill - Package tool for team reuse.
5. Share with team - Commit ``.claude/skills/your-tool/``.

****Database Migration Tool:****

`` ```

Create a CLI tool using Typer for managing database operations:

Commands:

- db migrate - run pending Alembic migrations
- db rollback - rollback last migration
- db status - show migration state
- db seed - populate with test data from fixtures/

Requirements:

- Proper type hints throughout
- Rich console output with colors and tables
- `--dry-run` flag for safe testing
- Environment-aware (dev/staging/prod)
- Follows our existing patterns in scripts/

Put it in `scripts/db_cli.py` and add to `pyproject.toml` scripts

`` ```

Tooling Generation: Readiness Checklist

- ✓ **I/O specification with concrete examples** – Exact input format, expected output, error responses – include 2–3 input/output examples, not just descriptions.
- ✓ **Runtime environment documented** – OS, language version, available deps, execution context (CI, cron, manual) – prevents Claude from using unavailable packages.
- ✓ **Existing scripts as reference patterns** – Point to scripts/ directory – Claude matches error handling, logging style, and argument parsing conventions from examples.
- ✓ **Spec-driven approach for complex tools** – Create requirements.md + design.md + tasks.md before generation – spec serves as contract both human and AI reference (2025 best practice).
- ✓ **--help documentation standard** – Every tool must include usage docs – add this to your prompt template; tools are only valuable if teammates can discover how to use them.
- ✓ **Skills directory for team distribution** – .claude/skills/ structure ready; generate → test → refine → document → package as skill → commit for reuse.
- ✓ **CLI discovery pattern** – AI can learn unfamiliar CLI tools via --help – prompt: “Use foo-tool --help to learn about it, then use it to solve X”.

Working with Git

- Automate commit workflows - stage changes, create commits with conventional message formats, and push.
- Run parallel development sessions with git worktrees.
- Create and manage pull requests.
- Resolve Merge Conflicts.

Working with Git

```

I need to work on two features simultaneously:

1. feature/async-refactor (current work)
2. hotfix/db-connection-leak (urgent fix needed)

Set up a git worktree for the hotfix so I can switch between them without stashing. Explain the workflow.

```

```

Review my staged changes and create a commit message that:

- Follows conventional commits (feat/fix/refactor/etc.)
- Has a clear, specific subject line under 72 chars
- Includes body explaining WHY these changes were made
- References any related issues



Don't commit yet - show me the message first.

```

```

Create a pull request for this branch:

- Generate title from commits (conventional format)
- Write description summarizing all changes
- Include "## Changes" section with bullet points
- Add "## Testing" section describing how to test
- Include "## Type Checking" - confirm mypy passes
- Reference related issues if commit messages mention them

Use GitHub CLI (gh pr create).

```

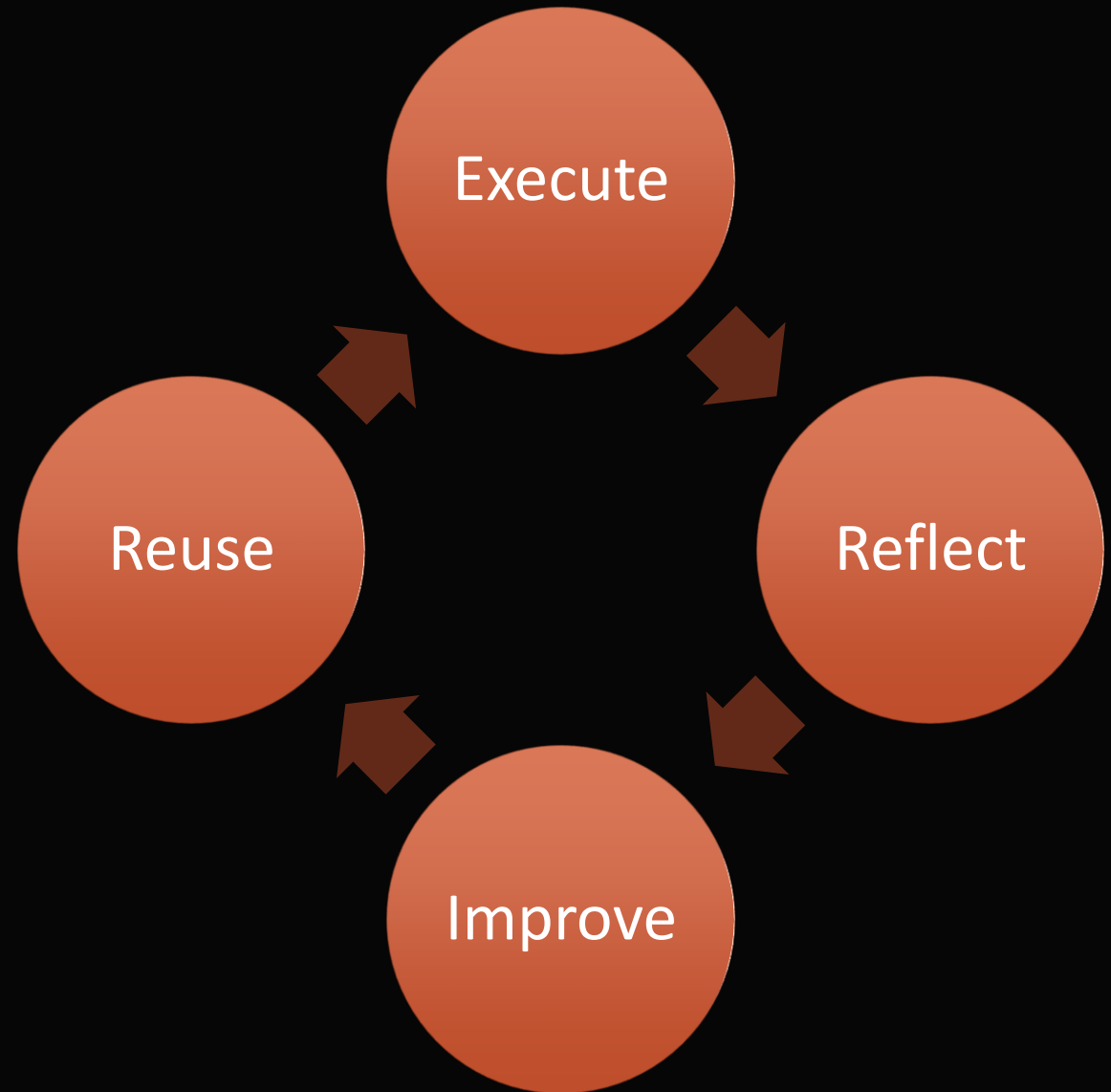
Working with Git:

Readiness Checklist

- ✓ **Commit convention in CLAUDE.md** – Format, types (feat/fix/refactor), scope, tense (imperative), ticket references – Claude generates excellent messages when the format is defined.
- ✓ **Atomic commit discipline** – One concern per commit – AI tends toward giant commits; instruct: “commit each logical change separately”.
- ✓ **PR template with structured sections** – .github/PULL_REQUEST_TEMPLATE.md: Summary, Test Plan, Breaking Changes – Claude fills these automatically from diff analysis.
- ✓ **gh CLI authenticated** – Claude uses gh for PR creation, review, issue management – verify gh auth status before the session; zero extra setup needed.
- ✓ **allowedTools configured for git safety** – Pre-approve safe ops (git status, diff, log) in settings.json – Claude refuses force-push/hard-reset by default; don’t override this.
- ✓ **Pre-commit hooks active** – Lint + format + test checks – catches issues in AI-generated code before they reach remote; mechanical consistency enforcement.
- ✓ **CODEOWNERS file configured** – Auto-assigned reviewers based on file ownership – ensures AI-generated changes get reviewed by the right humans.

💡 Evolve Your Claude Code Setup Incrementally

- **Week 1**
Single workflow (e.g., pytest), Basic `CLAUDE.md`, one slash command.
- **Week 2**
Add second workflow, Expand instructions, add skill.
- **Week 3**
Optimize and customize, Refine based on actual usage.
- **Week 4+**
Team sharing, Document and share what works.



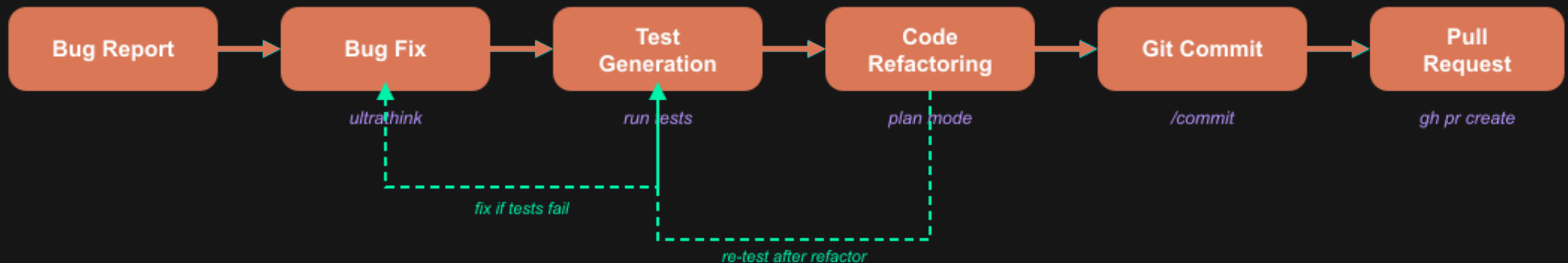
[cc-handbook/fundamentals/incrementally-evolve-setup](#)

Workflow Chaining

The 7 skills from this session are not isolated – they form a natural pipeline in daily development. A single Claude Code session can chain multiple workflows together.

Daily Workflow: From Bug Report to Pull Request

Single Claude Code Session



Today we've learned:

- AI-Assisted Use Cases

What's next?

Advanced Development Workflows